

0.23. Sans « x = » : la console range la position

Le 0.20 sans les « x = » : `deplace_x(x, ...)`; seul sur sa ligne, et x change quand même. Les nombres le montrent.

1. Ce qu'il faut comprendre

C'est le 0.20, avec un seul changement : les deux `x =` ont disparu.

Ce qui est nouveau ici : quand l'appel est **seul sur sa ligne** et que la position donnée est une **variable** (`x`), la console **range elle-même** la position d'arrivée dans cette variable. Le compilateur réécrit la ligne en `x = deplace_x(x, ...)` : c'est exactement le 0.20, la même cartouche, octet pour octet.

La preuve : les nombres affichent toujours `005` puis `000`. Personne n'a écrit `x =`, et pourtant `x` a changé.

C'est une exception, réservée à la console. La règle du C ne change pas : une fonction reçoit une copie. Seules les fonctions de la console (`deplace_x`, `deplace_y`, et celles qui viendront) ont droit à ce rangement. Pour **tes** fonctions, il faut toujours `x =` (le 0.33 le montre).

Une condition : la position doit être une **variable**. `deplace_x(3, 0, ALPHABET[0], 5)` fait bouger la lettre, mais il n'y a nulle part où ranger la colonne.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1637 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;      // +5 : avance ; -5 : recule
```

2. UNE VARIABLE — X

```
uint8_t x = 0;        // la colonne de la lettre
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

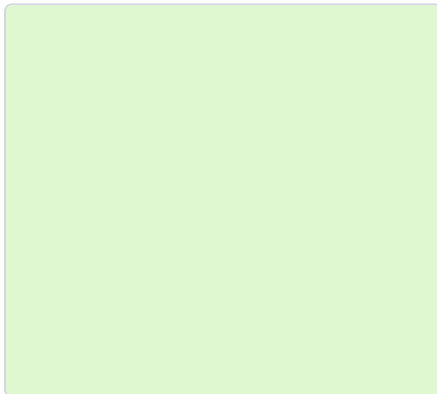
```
int main() {
    // Le changement : plus de « x = » devant deplace_x.
    //
    //   deplace_x(x, 0, ALPHABET[0], pas);
    //           |
    //           +-- x est une VARIABLE : la console y range toute seule la
    //               colonne d'arrivée. Le compilateur réécrit la ligne en
    //               x = deplace_x(x, 0, ALPHABET[0], pas);
    //               C'est le 0.20, écrit plus court.
    deplace_x(x, 0, ALPHABET[0], pas);    // x vaut 5 quand même...
    nombre(0, 2, x);                     // ...la preuve : 005

    deplace_x(x, 0, ALPHABET[0], -pas);  // x vaut 0
    nombre(0, 2, x);                     // 000

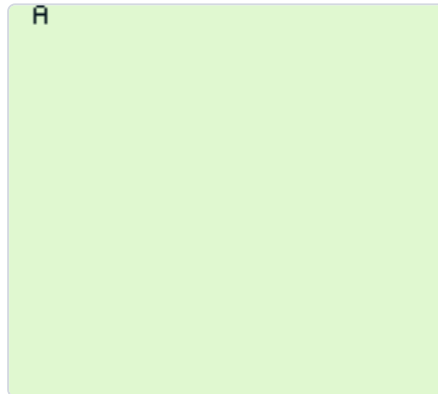
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

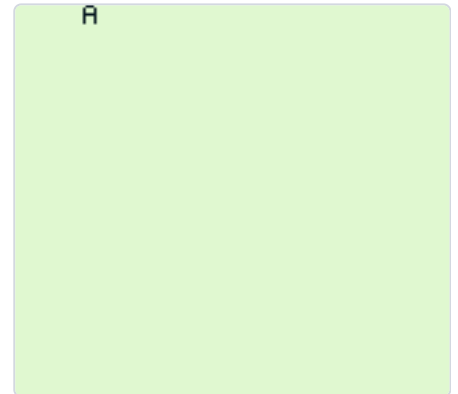
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



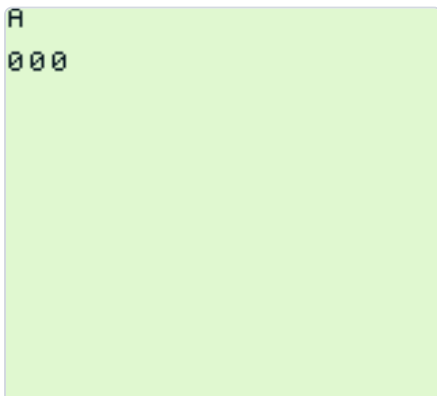
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Exactement le 0.20 : le A avance, 005 s'écrit ; il revient, 000 s'écrit.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ sans « x = », x a reçu 5 : 005 affiché

✓ puis 0 : 000 affiché (x = 0)

✓ le A est revenu en colonne 0

Ensuite : **0.23.1. Sans « x = » : la console range la position — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++